



ROUTEZONE

Prédiction de la gravité des accidents routiers

RAPPORT PROFESSIONNEL — ÉPREUVE E2

Veille technologique, benchmark et POC

Candidate	Meriem Abdelouahed
Formation	Développeur en Intelligence Artificielle — Simplon × Microsoft
Certification	RNCP37827
Soutenance	Juin 2026 — distanciel
Bloc / Compétences	Bloc 2 — Compétences C6 à C8
Version	1.0 — 18 mai 2026 (document évolutif, mis à jour jusqu'à la soutenance)

Sommaire

1. Présentation du projet et de la problématique.....	2
2. Dispositif de veille technologique.....	3
3. Mon cheminement d'apprentissage.....	4
4. Benchmark des services d'Intelligence Artificielle.....	9
5. Justification de mon choix technique.....	13
6. POC RouteZone : un projet industrialisé.....	14
7. Bilan et perspectives.....	17
Annexes.....	18

1. Présentation du projet et de la problématique

1.1 Présentation de RouteZone

RouteZone est un projet en machine learning capable de prédire la gravité des accidents routiers (Grave : tué ou hospitalisé / Pas grave : indemne ou blessé léger). Ce modèle repose sur les données BAAC 2022-2024 disponibles sur le site du Ministère de l'Intérieur, publiées en open data.

L'objectif de RouteZone est de servir d'aide à la décision pour les forces de l'ordre et les services de secours, afin de prioriser les interventions sur les cas les plus critiques. Le modèle s'appuie également sur les temps réels d'intervention des services de secours (pompiers et services d'urgences), calculés via le moteur de routage OSRM, ce qui permet de mieux mesurer l'état d'urgence de chaque accident.

1.2 Pourquoi une veille technologique ?

Le secteur du Machine Learning évolue très rapidement. Une veille technologique permet d'identifier les innovations, de comprendre les nouvelles tendances, et de s'adapter pour rester pertinent dans ses choix techniques. De nouveaux algorithmes apparaissent régulièrement, les bonnes pratiques évoluent (gestion du déséquilibre des classes, métriques d'évaluation, méthodes de calibration), et de nouveaux outils sont mis à disposition.

Pour se tenir à la page et faire les bons choix techniques, il est indispensable de tester et de comparer les solutions disponibles. C'est exactement la démarche que j'ai suivie pour le projet RouteZone.

1.3 Plan du rapport

Ce rapport se présentera de la manière suivante :

- Mon dispositif de veille technologique
- Mon cheminement d'apprentissage à travers les notions clés du projet
- Le benchmark des solutions IA disponibles sur le marché
- La justification de mes choix techniques
- La mise en œuvre concrète sur RouteZone (POC)
- Le bilan et les perspectives d'évolution

2. Dispositif de veille technologique

2.1 Mes sources de veille

Pour réaliser ma veille technologique, je me suis appuyée sur plusieurs types de sources :

- **Sites de référence** : DataCamp, La revue IA (Ilyes Talbi), Bureau des Talents, documentations officielles comme celle de scikit-learn
- **Recherches ciblées** : sur des moteurs de recherche classiques (Google) quand j'ai besoin d'une réponse précise sur un sujet
- **Vidéos YouTube** : Machine Learnia (Guillaume Saint-Cirgue) et LeCoinStat, qui vulgarisent le monde de la data
- **Agrégateurs** : Daily.dev, principalement en anglais. Quand je cherche des informations sur des méthodes récentes, la documentation se trouve surtout dans cette langue (je traduis les pages quand nécessaire pour mieux comprendre)
- **Accompagnement IA** : j'utilise principalement Claude (Anthropic) pour structurer mes réflexions, comprendre des concepts difficiles, et reformuler ce que je lis afin d'approfondir certaines notions

2.2 Mes outils d'organisation

Pour m'organiser dans ma veille technologique, j'utilise plusieurs outils :

- **Un carnet de bord** (docs/actualite.md) qui documente les articles lus, mes notes de compréhension, les liens importants
- **Notion** pour mes notes personnelles au quotidien (sous forme de brouillon) avant de mettre au propre dans le carnet de bord
- **Un fichier wins.md** qui liste les tâches que j'ai réalisées au quotidien, afin de garder une trace écrite du cheminement de mon projet
- **GitHub Projects** : j'ai mis en place un Kanban qui liste toutes les tâches à faire étape par étape pour une organisation optimale du projet (cf. capture d'écran section méthodologie agile du document SPECS.md)

2.3 Ma méthode de veille

Pour m'informer sur le monde de l'informatique et du Machine Learning, j'ai mis en place une fréquence hebdomadaire d'un ou deux articles à lire. Je sélectionne des articles sur des sources fiables en vérifiant que :

- L'auteur est identifié et reconnu dans le milieu
- Le contenu est récent
- Les références citées sont sérieuses

Lorsque j'apprends une nouvelle technologie (comme Grafana par exemple), je m'informe sur le concept général puis je le note dans mon Notion afin de garder une trace de mon apprentissage. Si un terme technique ou une notion est difficile à comprendre, je demande à Claude de me l'expliquer en vulgarisant au maximum, par exemple via une analogie. Cette méthode m'a été très utile pour comprendre la différence entre une bibliothèque et un framework, ou encore pour saisir la nuance entre un score et une probabilité empirique.

Cette méthode de travail m'a permis d'avancer sereinement sur mon projet, car j'ai tendance à vouloir comprendre un concept de A à Z avant de l'utiliser.

2.4 Limites et honnêteté intellectuelle

Ma veille s'est réellement structurée à partir de mi-mai 2026, dans la phase de consolidation du projet. Avant cela, je gardais mes notes sur Notion en brouillon, car je préférais d'abord avancer sur le développement du projet. À ce stade de mon parcours, j'ai enfin voulu mettre au propre tout ce que j'avais appris, plutôt que de le faire au fur et à mesure.

Cette démarche tardive de formalisation est assumée : elle correspond à ma façon de travailler, où je préfère d'abord comprendre et construire, puis documenter ensuite avec recul.

Ce rapport est par nature évolutif : la veille technologique ne s'arrête pas à sa date de rédaction. Le carnet de bord docs/actualite.md est mis à jour à chaque nouvelle lecture significative jusqu'à la soutenance.

3. Mon cheminement d'apprentissage

3.1 Méthode d'apprentissage assistée par IA

Pour ce projet, j'ai utilisé Claude (Anthropic) comme outil pédagogique d'accompagnement. Cet outil m'a permis de poser des questions précises sur des concepts difficiles, d'obtenir des explications adaptées à mon niveau, de reformuler avec mes propres mots pour valider ma compréhension, et de travailler de manière autonome quand mes formateurs n'étaient pas disponibles.

Mes notes de cheminement sont archivées sur Notion sous forme de conversations questions/réponses. Cette approche illustre une compétence essentielle en 2026 : savoir utiliser les outils d'IA comme accélérateurs d'apprentissage, tout en validant l'information par recoupement avec des sources documentaires (articles, vidéos, documentation officielle).

Voici trois encadrés qui présentent mes principaux apprentissages, en croisant les explications de Claude avec des recherches autonomes sur le web.

3.2 Encadré 1 — Comprendre le Gradient Boosting et LightGBM

Question initiale : « Pourquoi avoir choisi LightGBM plutôt qu'un autre algorithme comme XGBoost ou Random Forest ? »

Démarche d'apprentissage

1. J'ai d'abord demandé à Claude (Anthropic) de m'expliquer le concept général de boosting avec des analogies simples (mes notes Notion documentent cet échange).
2. J'ai consulté une vidéo YouTube sur le sujet (« Comprendre le Gradient Boosting Simplement », lien dans actualite.md). Cependant l'exemple utilisé étant un cas de régression (prédiction de prix immobiliers), j'ai préféré chercher une source adaptée à mon cas d'usage (classification binaire).
3. J'ai lu l'article « LightGBM : mieux que XGBoost ? » de Nada Belaidi sur Blent.ai (25 février 2022).

Ce que j'en ai retenu

- Le Gradient Boosting est une méthode où plusieurs modèles « faibles » (généralement des arbres de décision) sont entraînés séquentiellement. Chaque nouvel arbre corrige les erreurs du précédent.
- LightGBM est une bibliothèque développée par Microsoft (2016) qui implémente le Gradient Boosting. Sa particularité : il fait croître ses arbres verticalement (par feuille) là où XGBoost les fait croître horizontalement (par niveau).
- LightGBM est plus rapide et plus performant sur les grands datasets que XGBoost, grâce à deux techniques d'optimisation : GOSS (Gradient-based One-Side Sampling) qui concentre l'apprentissage sur les exemples mal prédits, et EFB (Exclusive Feature Bundling) qui regroupe les features incompatibles pour économiser la mémoire.

Application sur RouteZone

LightGBM est particulièrement adapté à mon projet car j'ai un grand volume de données (153 054 accidents) et des données tabulaires hétérogènes (numériques + catégorielles). Le paramètre `max_depth` est réglé à 10 pour limiter la profondeur des arbres et éviter le surapprentissage (overfitting).

Limites de mes lectures

Certains concepts techniques évoqués dans l'article (algorithme CART, indice de Gini, GBDT) restent à approfondir dans une future veille. Je retiens néanmoins l'essentiel : ce sont des outils statistiques utilisés en interne pour mesurer la qualité d'une séparation dans un arbre de décision.

Démarche critique

Cette section illustre l'importance de savoir trier l'information en veille technologique. Une vidéo populaire n'est pas toujours adaptée à son contexte d'usage. La triangulation des sources (Claude + vidéo + article) m'a permis de valider ma compréhension.

3.3 Encadré 2 — Docker et la conteneurisation

Question initiale : « Comment garantir que mon projet RouteZone fonctionne de la même manière sur n'importe quel ordinateur, que ce soit le mien, celui d'un formateur, ou un serveur de production ? »

Démarche d'apprentissage

1. J'ai d'abord utilisé Docker pendant le développement sans bien comprendre tous les concepts (compétence « faire fonctionner » avant « comprendre »).
2. Pour ma veille, j'ai lu l'article « Docker pour les débutants : Guide pratique des conteneurs » de Moez Ali (data scientist, fondateur de PyCaret) publié sur DataCamp et mis à jour le 24 février 2025.

Critères de fiabilité

DataCamp est une plateforme de formation Data Science reconnue mondialement. L'auteur Moez Ali est lui-même créateur d'une bibliothèque ML open source (PyCaret). Article récent, technique mais pédagogique.

Ce que j'en ai retenu

Docker est un outil qui permet d'empaqueter un projet (code, dépendances, configuration) de manière standardisée. Il résout définitivement le problème du « ça ne marche pas sur mon PC ».

Pour bien comprendre, j'ai retenu trois concepts avec leurs analogies :

Concept Docker	Analogie	Rôle
Image	Un CD	Modèle en lecture seule qui contient l'OS, le code et les dépendances
Container	Un lecteur CD	Lit l'image de manière isolée pour exécuter l'application
Docker Hub	Une bibliothèque de CDs	Système pour stocker et partager les images

Une image se construit grâce à un Dockerfile : un script qui automatise toute la création de l'environnement. Une fois construite, elle peut être lue par un container pour exécuter l'application dans un environnement isolé.

Quand plusieurs containers doivent fonctionner ensemble (par exemple une API qui parle à une base de données), on utilise Docker Compose avec un fichier docker-compose.yml qui relie plusieurs services.

Application sur RouteZone

Mon projet utilise 5 containers Docker orchestrés avec Docker Compose :

- **routezone-api-data** : l'API REST des données (FastAPI)
- **routezone-api-ia** : l'API REST de prédiction (FastAPI + LightGBM)
- **routezone-db** : la base de données PostgreSQL 16
- **routezone-prometheus** : le système de monitoring
- **routezone-grafana** : les tableaux de bord de monitoring

Pour lancer toute l'application, une seule commande suffit : `docker-compose up -d`. Pour l'arrêter : `docker-compose down`.

Limites de mes lectures

Certains concepts avancés comme les modes de réseau (overlay, macvlan) ou l'orchestration à grande échelle (Kubernetes) n'ont pas été approfondis car ils dépassent les besoins de mon projet. À explorer dans une future veille si je travaille sur des architectures distribuées.

3.4 Encadré 3 — MLflow et le suivi d'expériences ML

Question initiale : « Comment garder une trace de tous les modèles que je teste, comparer leurs performances, et retrouver facilement le meilleur ? »

Démarche d'apprentissage

1. J'ai utilisé MLflow dans mon notebook 05 pour tracker mes expériences de modélisation (LogisticRegression, RandomForest, XGBoost, LightGBM).
2. Pour ma veille, j'ai lu l'article « Qu'est-ce que MLflow ? » d'Alexandre Scheck publié sur Bureau des Talents (décembre 2025).

Critères de fiabilité

Bureau des Talents est un site de référence sur les métiers de la tech. L'auteur est identifié et l'article est récent (moins de 6 mois). Article cohérent avec la documentation officielle de MLflow.

Ce que j'en ai retenu

MLflow est une plateforme open source créée par Databricks pour gérer le cycle de vie des projets de Machine Learning. Sans MLflow, on perd vite le fil de ses expériences : quel modèle a donné quel score ? avec quels paramètres ? sur quel jeu de données ?

MLflow apporte une traçabilité sur les modèles créés : performances, scores, hyperparamètres. Les résultats peuvent ensuite être comparés avec des graphiques et des tableaux dans l'interface MLflow.

Les principales utilisations :

- Suivre les différentes versions d'un modèle au fil des essais
- Comparer les performances de plusieurs expériences
- Stocker les paramètres d'entraînement et les jeux de données utilisés
- Enregistrer et déployer facilement les modèles entraînés

MLflow est composé de 4 modules indépendants :

- **MLflow Tracking** : enregistre et visualise les résultats d'expériences
- **MLflow Projects** : structure les projets avec des fichiers standardisés
- **MLflow Models** : gère les modèles entraînés et les rend portables
- **MLflow Registry** : stocke et versionne les modèles validés

Application sur RouteZone

J'ai principalement utilisé MLflow Tracking dans mon notebook 05_modelisation. Pour chaque modèle testé (LogisticRegression, RandomForest, XGBoost, LightGBM), j'ai loggé :

- Les hyperparamètres (max_depth, n_estimators, learning_rate)
- Les métriques (Recall, Precision, F1, AUC-ROC, accuracy)
- Le modèle entraîné (artefact .pkl)
- Le seed pour la reproductibilité

L'interface MLflow UI m'a permis de comparer visuellement les 4 modèles côte à côte et de confirmer que LightGBM offrait le meilleur compromis (Recall GRAVE de 0.80 lors de la phase d'entraînement initial).

Cette traçabilité est précieuse pour ma soutenance : je peux prouver mes choix avec des données chiffrées, pas juste des intuitions.

Pour approfondir

Le module MLflow Registry permettrait de gérer plusieurs versions de mon modèle en production (V1, V2, V3 OSRM brut). C'est une piste d'évolution pour une V2 du projet.

4. Benchmark des services d'Intelligence Artificielle

4.1 Objectif du benchmark

Pour faire un choix éclairé sur la stack technique de RouteZone, j'ai comparé plusieurs solutions disponibles sur le marché pour entraîner et déployer un modèle de Machine Learning. Le benchmark se concentre sur 5 solutions représentatives :

- **Azure Machine Learning** (Microsoft) — cloud
- **AWS SageMaker** (Amazon) — cloud
- **Google Vertex AI** (Google) — cloud
- **Hugging Face** — cloud spécialisé open source
- **LightGBM self-hosted** (ma solution) — local / on-premise

4.2 Critères de comparaison

J'ai retenu 8 critères pertinents pour un projet d'IA en 2026 :

Critère	Pourquoi c'est important
Coût	Pour un projet de certification sans budget, c'est un facteur déterminant
Courbe d'apprentissage	Temps nécessaire pour devenir productif sur la plateforme
Performance	Capacité à entraîner et servir des modèles rapidement
Scalabilité	Capacité à monter en charge si le nombre d'utilisateurs explose
Souveraineté des données	Où sont stockées les données (RGPD, conformité légale)
Maîtrise technique	Le service cache-t-il la complexité ou nous oblige-t-il à tout comprendre ?
Écosystème	Intégration avec d'autres outils (Docker, MLflow, etc.)
Monitoring intégré	Outils de surveillance fournis ou à configurer soi-même

4.3 Présentation des solutions

Azure Machine Learning

Éditeur : Microsoft. Type : Plateforme cloud d'IA managée.

Azure ML est une plateforme complète qui couvre tout le cycle de vie d'un projet Machine Learning : préparation des données, entraînement des modèles, déploiement, monitoring. Elle propose une interface graphique (Azure ML Studio) en plus d'un SDK Python.

Points forts

- Intégration native avec MLflow (le tracking est inclus, pas besoin de configurer un serveur)
- Application Insights intégré pour le monitoring
- Pipelines automatisés (AutoML)
- Conformité RGPD et hébergement en Europe possible

Points faibles

- Coût élevé pour de gros volumes de calcul
- Courbe d'apprentissage relativement raide (concepts spécifiques à Azure : Workspace, Compute Cluster, etc.)
- Dépendance forte à l'écosystème Microsoft

Cas d'usage idéal

Entreprise déjà sur Azure, projet en production avec besoin de scalabilité automatique.

AWS SageMaker

Éditeur : Amazon Web Services. Type : Plateforme cloud d'IA managée.

SageMaker est le service Machine Learning d'AWS, le leader mondial du cloud. Comme Azure ML, il couvre tout le cycle de vie ML mais avec des spécificités AWS.

Points forts

- Très large catalogue d'instances de calcul (CPU, GPU, IPU)
- Forte intégration avec les autres services AWS (S3 pour le stockage, Lambda pour le serverless)
- CloudWatch pour le monitoring intégré
- SageMaker JumpStart : modèles pré-entraînés prêts à l'emploi

Points faibles

- Tarification complexe et difficile à anticiper
- Documentation parfois confuse (beaucoup de services qui se recouvrent)
- Lock-in fort sur l'écosystème AWS
- Souveraineté des données : serveurs principalement aux États-Unis (régions UE disponibles mais à configurer)

Cas d'usage idéal

Startup ou grande entreprise déjà sur AWS, projets nécessitant beaucoup de GPU.

Google Vertex AI

Éditeur : Google Cloud Platform. Type : Plateforme cloud d'IA managée.

Vertex AI est la plateforme unifiée d'IA de Google, qui regroupe les anciens services AI Platform et AutoML. Elle bénéficie de la recherche Google en IA (Gemini, TensorFlow).

Points forts

- Excellente intégration avec TensorFlow (créé par Google)
- AutoML très avancé (Google a une grande expertise sur l'automatisation ML)
- Bons outils de monitoring (Cloud Monitoring)
- Accès facile aux modèles Gemini pour l'IA générative

Points faibles

- Coût élevé (positionné sur le haut de gamme)
- Moins répandu en entreprise que Azure et AWS
- Souveraineté des données : serveurs principalement aux États-Unis

Cas d'usage idéal

Projets d'IA générative ou de deep learning avec TensorFlow.

Hugging Face

Éditeur : Hugging Face (start-up franco-américaine). Type : Plateforme cloud open source spécialisée.

Hugging Face est devenu la référence mondiale pour les modèles open source de Machine Learning, surtout en NLP (traitement du langage). La plateforme propose un hub de modèles pré-entraînés (BERT, GPT, Llama, etc.) et un service d'hébergement de modèles (Inference API).

Points forts

- Communauté open source très active
- Catalogue immense de modèles pré-entraînés (plus de 500 000 modèles)
- Inference API simple à utiliser
- Tarification claire et accessible (gratuit pour les petits volumes)

Points faibles

- Principalement orienté NLP et Deep Learning (moins adapté pour les modèles tabulaires comme LightGBM)
- Monitoring limité comparé aux 3 grands clouds
- Moins de fonctionnalités d'entreprise (gouvernance, conformité)

Cas d'usage idéal

Projets NLP, prototypage rapide avec des modèles pré-entraînés.

LightGBM self-hosted (solution retenue)

Éditeur : Microsoft (LightGBM est open source). Type : Solution locale / on-premise.

LightGBM est une bibliothèque Python open source que j'utilise directement sur ma machine, sans passer par un service cloud. L'entraînement et la prédiction se font en local, et le modèle est servi via une API FastAPI conteneurisée avec Docker.

Points forts

- Coût zéro (open source, pas d'abonnement cloud)
- Souveraineté totale des données (rien ne sort de ma machine)
- Apprentissage complet de la stack (je comprends toute la chaîne)
- Pas de lock-in vendeur
- Performance excellente sur données tabulaires

Points faibles

- Scalabilité limitée (dépend de ma machine)
- Monitoring à configurer soi-même (Prometheus + Grafana)

- Pas de calcul distribué facile
- Disponibilité limitée (ma machine doit être allumée)

Cas d'usage idéal

Projet de certification, POC, données sensibles, équipe maîtrisant la stack technique.

4.4 Tableau comparatif synthétique

Critère	Azure ML	AWS SageMaker	Vertex AI	Hugging Face	LightGBM local
Coût	Élevé	Élevé	Très élevé	Faible à modéré	Gratuit
Courbe d'apprentissage	Raide	Raide	Raide	Modérée	Modérée
Performance	Excellente	Excellente	Excellente	Bonne (NLP)	Excellente (tabulaire)
Scalabilité	Très bonne	Très bonne	Très bonne	Bonne	Limitée
Souveraineté données	UE possible	US (UE option)	US principal	US/Europe	Totale (local)
Maîtrise technique	Cachée	Cachée	Cachée	Partielle	Totale
Écosystème	Microsoft	AWS	Google	Open source	Universel (Docker)
Monitoring intégré	Oui (App Insights)	Oui (CloudWatch)	Oui	Limité	Non (à configurer)

4.5 Sources utilisées pour ce benchmark

Ce benchmark s'appuie sur les documentations officielles de chaque éditeur, ainsi que sur des articles comparatifs récents :

- **Azure ML** : learn.microsoft.com/fr-fr/azure/machine-learning/
- **AWS SageMaker** : docs.aws.amazon.com/sagemaker/
- **Google Vertex AI** : cloud.google.com/vertex-ai/docs
- **Hugging Face** : huggingface.co/docs
- **LightGBM** : lightgbm.readthedocs.io/

Limites du benchmark

Cette comparaison est principalement théorique, basée sur la documentation officielle et l'expérience que j'ai acquise via ma certification Microsoft AI-900. Je n'ai pas testé personnellement chacune de ces plateformes en profondeur. Pour un benchmark plus poussé, il faudrait réaliser un POC sur chaque solution avec un même jeu de données et mesurer les performances réelles (temps d'entraînement, coût total, qualité du modèle).

5. Justification de mon choix technique

5.1 Le contexte de ma décision

Pour le développement de RouteZone, j'avais plusieurs options pour entraîner et déployer mon modèle : utiliser un service cloud d'IA (Azure Machine Learning, AWS SageMaker, Google Vertex AI) ou travailler en local sur ma propre machine. J'ai choisi de travailler en local.

Les services cloud présentent des avantages indéniables : scalabilité, calcul à la demande, gestion simplifiée des infrastructures, et facilité de travail collaboratif. Cependant, ils présentent aussi des contraintes : coûts d'abonnement, dépendance à un fournisseur tiers, et souveraineté des données qui ne sont plus stockées sur la machine de l'utilisateur.

5.2 Les raisons de mon choix

J'ai pris la décision de travailler en local pour les raisons suivantes :

- **Travail individuel** : je travaille seule sur mon serveur, donc le local est plus pratique. Le cloud aurait surtout du sens pour une équipe distribuée.
- **Coût zéro** : aucun abonnement à payer, contrairement aux services cloud qui facturent à l'usage (CPU, stockage, requêtes).
- **Souveraineté des données et RGPD** : toutes les données BAAC restent sur mon ordinateur. Je conserve un contrôle total sur leur traitement, sans transfert vers un acteur tiers.
- **Performance suffisante** : LightGBM est un algorithme léger qui ne nécessite pas de GPU. Il tourne très bien sur un CPU standard.
- **Apprentissage complet de la stack** : un service cloud cache une partie de la complexité technique. En travaillant en local, je suis confrontée à toute la chaîne (entraînement, sauvegarde, API, conteneurisation, monitoring), ce qui correspond à l'objectif pédagogique de ma formation.
- **Solution déjà industrialisée** : grâce à Docker, mon projet est portable et peut être déployé sur n'importe quel serveur (local, VPS, cloud) sans modification du code.

5.3 Les limites de mon choix

Travailler en local présente certains inconvénients qu'il convient d'assumer :

- **Évolutivité limitée** : l'augmentation de la capacité de stockage et de calcul nécessite généralement un cloud. Par exemple, j'ai pris la décision de ne traiter que 3 années (2022 à 2024) pour les 12 fichiers CSV BAAC, soit 153 054 accidents impliquant 413 570 usagers. Cette limite s'explique par la capacité de stockage et de calcul de ma machine. Avec un cloud, j'aurais pu traiter l'historique complet (10 années ou plus), ce qui aurait probablement amélioré la capacité de généralisation du modèle.
- **Monitoring moins avancé** : un cloud propose des outils de monitoring intégrés (CloudWatch sur AWS, Application Insights sur Azure). En local, j'ai dû mettre en place ma propre stack (Prometheus + Grafana), ce qui m'a demandé plus de configuration.
- **Disponibilité limitée** : mon application n'est accessible que lorsque ma machine est allumée. Pour un déploiement réel, un hébergement cloud (ou un VPS — Virtual Private Server, c'est-à-dire un serveur loué chez un hébergeur qui tourne 24h/24) serait indispensable.

Ces limites sont assumées dans le cadre d'un POC de certification. Pour une mise en production réelle, un déploiement cloud (ou hybride) serait à envisager.

6. POC RouteZone : un projet industrialisé

6.1 Objectif du POC

Par définition, un POC (Proof of Concept) est une démonstration de faisabilité d'un concept ou d'un projet. En d'autres termes, il sert à montrer concrètement comment j'ai mis en pratique ma veille technologique sur mon projet.

L'objectif de mon POC est de démontrer la prédiction de la gravité d'un accident (Grave ou Pas grave) à partir des données BAAC, avec une approche industrialisée intégrant API, application web, base de données, monitoring et tests automatisés.

6.2 Architecture globale et choix techniques

Frontend — Streamlit

J'ai choisi d'utiliser Streamlit pour le frontend de mon application. Streamlit est une bibliothèque Python open source qui facilite la création et le partage d'applications web personnalisées pour le Machine Learning et la Data Science. J'ai opté pour ce frontend car il est rapide, simple et personnalisable. En revanche, la personnalisation reste limitée contrairement à un code HTML/CSS pur qui offre une liberté totale de design.

Backend — FastAPI

J'ai fait le choix d'utiliser FastAPI à la fois pour l'API des données et pour l'API de prédiction (IA). FastAPI est un framework web Python récent (créé en 2018) destiné à la construction d'API, qui permet de connecter un service à un autre logiciel pour échanger des données et des fonctionnalités. J'en avais besoin pour connecter ma base de données PostgreSQL et mon modèle de prédiction LightGBM.

J'ai choisi FastAPI pour plusieurs raisons : c'est l'une des API Python les plus rapides du marché, elle augmente la productivité d'environ 40% selon les benchmarks officiels, la documentation Swagger est générée automatiquement, et la sécurité est privilégiée par défaut (validation des entrées via Pydantic, support natif de l'authentification JWT).

Base de données — PostgreSQL

J'ai sélectionné PostgreSQL pour ma base de données. C'est un système de gestion de base de données relationnelle orienté objet, puissant et open source, capable de prendre en charge en toute sécurité de gros volumes de données. C'est lui qui stocke les données et qui répond aux requêtes SQL.

Au départ, j'avais sélectionné SQLite car les deux solutions présentent des similitudes (gratuit, code source ouvert, intégration simple). Mais j'ai finalement basculé sur PostgreSQL car il propose une meilleure performance pour les grands volumes de données. À l'heure actuelle, SQLite aurait pu faire la même chose pour mon volume, mais en vue d'une mise en production future, PostgreSQL est plus adapté pour ce type de projet (concurrency, transactions ACID, support géospatial pour les coordonnées GPS).

ACID est un acronyme désignant 4 propriétés essentielles d'une base de données fiable : Atomicité (une opération s'exécute entièrement ou pas du tout), Cohérence (la base reste toujours dans un état valide), Isolation (plusieurs opérations simultanées ne s'entremêlent pas) et Durabilité (les données validées sont définitivement sauvegardées, même en cas de panne).

Modèle de prédiction — LightGBM

LightGBM s'est imposé naturellement comme modèle final. J'ai testé plusieurs modèles de classification (LogisticRegression, RandomForest, XGBoost) et les performances de LightGBM se démarquaient grandement. LightGBM affiche un Recall GRAVE de 0.7643, ce qui est un très bon score pour prédire la gravité des accidents (priorité métier de mon projet).

LightGBM a pour avantage de réentraîner les arbres de décision pour les améliorer au fur et à mesure, tout en limitant l'overfitting grâce au paramètre `max_depth`. Le modèle final est exporté au format `.pkl`, ce qui permet de produire un artefact réutilisable, importé comme une dépendance dans mon projet.

Monitoring — Python Logging, Prometheus et Grafana

Pour la partie monitoring (surveillance de l'application en temps réel), j'ai mis en place 3 outils complémentaires :

- **Python Logging** : retrace les événements importants dans un fichier de logs structurés
- **Prometheus** : collecte mes métriques en continu (latence, nombre de requêtes, taux d'erreur)
- **Grafana** : affiche les métriques sous forme de graphiques sur un tableau de bord

L'ensemble permet d'assurer la qualité de service en production et de détecter les anomalies avant qu'elles ne deviennent critiques.

Conteneurisation — Docker

Enfin, pour rendre mon projet exploitable sur n'importe quel ordinateur, j'ai sélectionné Docker. Docker est un outil qui permet d'empaqueter un logiciel avec tout ce dont il a besoin pour fonctionner (code, dépendances, configuration), pour que chaque utilisateur ait le même environnement. Docker est devenu un incontournable pour tout projet destiné à être partagé ou déployé en production.

6.3 Étapes de mise en œuvre (chronologie)

Les différentes étapes chronologiques de mon projet sont les suivantes :

- **Téléchargement, collecte et exploration** des 12 fichiers CSV de la BAAC (2022-2024)
- **Nettoyage** (gestion des NaN, doublons, types) et enrichissement (Open-Meteo pour la météo réelle, OSRM pour les temps d'intervention des secours — Golden Hour)
- **Modélisation comparative avec MLflow** : 4 modèles testés (LogisticRegression, RandomForest, XGBoost, LightGBM)
- **Sélection et optimisation** du modèle final : LightGBM retenu, tuning manuel puis Optuna, exploration de la calibration des probabilités (cf. doc d'incident calibrator)
- **Industrialisation** : développement de l'API FastAPI, de l'application Streamlit, de la base PostgreSQL, et conteneurisation avec Docker
- **Monitoring et tests** : intégration de Prometheus, Grafana, Python Logging et pytest, avec CI/CD via GitHub Actions

6.4 Résultats obtenus

Voici les performances du modèle final (V3 OSRM brut, sans calibration) sur le jeu de test 2024 :

Métrique	Valeur
Recall GRAVE	0.7643
Precision GRAVE	0.4166
F1 macro	0.6956
AUC-ROC	0.8558
Accuracy	0.7760

Autres indicateurs de qualité du POC :

- **34 tests pytest verts** sur 34
- **CI/CD GitHub Actions opérationnelle** (tests automatiques à chaque push sur master)
- **5 containers Docker** orchestrés via docker-compose
- **Documentation complète** : README, rapports E1/E2, SPECS.md, doc d'incident calibrator

6.5 Validation du POC

Le POC RouteZone est fonctionnel et démontre qu'une approche d'aide à la décision en sécurité routière est techniquement réalisable. Les résultats obtenus (Recall GRAVE de 76%) sont supérieurs aux approches classiques sur ce dataset (cf. comparaison avec l'article d'Ilyes Talbi en section 3).

Le POC est prêt à être présenté à un commanditaire (autorités, services de secours, collectivités) pour validation métier et envisager une mise en production réelle.

7. Bilan et perspectives

7.1 Bilan de la veille technologique

La veille technologique a été un facteur clé dans la réussite de mon projet RouteZone. Sans elle, j'aurais probablement reproduit les choix techniques d'autres projets sans les comprendre, et je n'aurais pas su justifier mes décisions face à un jury ou un commanditaire.

Cette veille m'a permis :

- D'identifier les technologies pertinentes pour mon projet (LightGBM, FastAPI, Docker, MLflow, Prometheus, Grafana)
- De comprendre les concepts sous-jacents (Gradient Boosting, conteneurisation, tracking ML)
- De me positionner par rapport à l'existant (comparaison avec l'article d'Ilyes Talbi)
- De justifier mes choix par des sources fiables et récentes

7.2 Bilan du benchmark et du POC

Le benchmark des services IA m'a confirmé que la solution LightGBM self-hosted avec Docker était la plus adaptée à mon contexte (projet de certification, données sensibles, apprentissage complet de la stack). Les solutions cloud (Azure, AWS, GCP) seraient à privilégier pour un projet en production avec un fort besoin de scalabilité.

Le POC RouteZone démontre la faisabilité technique du projet. Les résultats sont au rendez-vous (Recall GRAVE de 76%, 34 tests verts, CI/CD opérationnelle), et l'ensemble est documenté et reproductible.

7.3 Perspectives d'évolution

Pour aller plus loin, plusieurs pistes d'évolution sont envisagées :

Court terme (V1.1)

- Compléter la veille technologique avec d'autres articles sur les sujets non encore approfondis (calibration, monitoring avancé, MLflow Registry)
- Améliorer la documentation des fonctionnalités existantes
- Ajouter un test pytest mesurant le Recall global pour détecter les régressions de performance

Moyen terme (V2)

- Explorer d'autres méthodes de calibration des probabilités (prefit sigmoid + seuil 0.35) pour améliorer la lisibilité des scores
- Mettre en place MLflow Registry pour gérer plusieurs versions du modèle en production
- Implémenter une politique de rétention automatique des données (purge des anciennes prédictions)
- Ajouter une pagination de l'historique des prédictions

Long terme (V3)

- Déployer l'application sur un VPS ou un cloud (Scaleway, OVH) pour assurer une disponibilité 24h/24
- Développer une application mobile native pour les forces de l'ordre
- Connecter RouteZone en temps réel aux systèmes d'information des services de secours
- Mettre à jour automatiquement le dataset BAAC à chaque nouvelle publication officielle

7.4 Conclusion

Ce rapport présente l'ensemble de ma démarche de veille technologique, de benchmark et de mise en œuvre concrète sur le projet RouteZone. Il témoigne d'une démarche structurée, où chaque choix technique est justifié par une compréhension réelle des enjeux et des alternatives disponibles.

La veille technologique n'est pas une activité ponctuelle mais un processus continu. Ce rapport sera enrichi jusqu'à la soutenance avec les nouvelles lectures et apprentissages.

Annexes

Liens et ressources

- **Repository GitHub** : github.com/Meriem69/Routezone
- **Carnet de veille** : [docs/actualite.md](#)
- **Document SPECS** : [docs/SPECS.md](#)
- **Doc d'incident calibrator** : [docs/incidents/incident_calibrator.md](#)

Articles consultés (synthèse)

Source	Titre	Auteur	Date
Blent.ai	LightGBM : mieux que XGBoost ?	Nada Belaidi	25/02/2022
DataCamp	Docker pour les débutants : Guide pratique des conteneurs	Moez Ali	24/02/2025
Bureau des Talents	Qu'est-ce que MLflow ?	Alexandre Scheck	12/2025
La revue IA	Comment gérer le déséquilibre des classes en machine learning	Ilyes Talbi	08/03/2021
La revue IA	XGBoost vs Random Forest : prédire la gravité d'un accident	Ilyes Talbi	06/09/2020
Mon Shot de Data Science	Précision et Rappel : Arrête de te tromper !	Non identifié	Non identifié

Les détails complets des articles et leur analyse critique sont disponibles dans [docs/actualite.md](#).